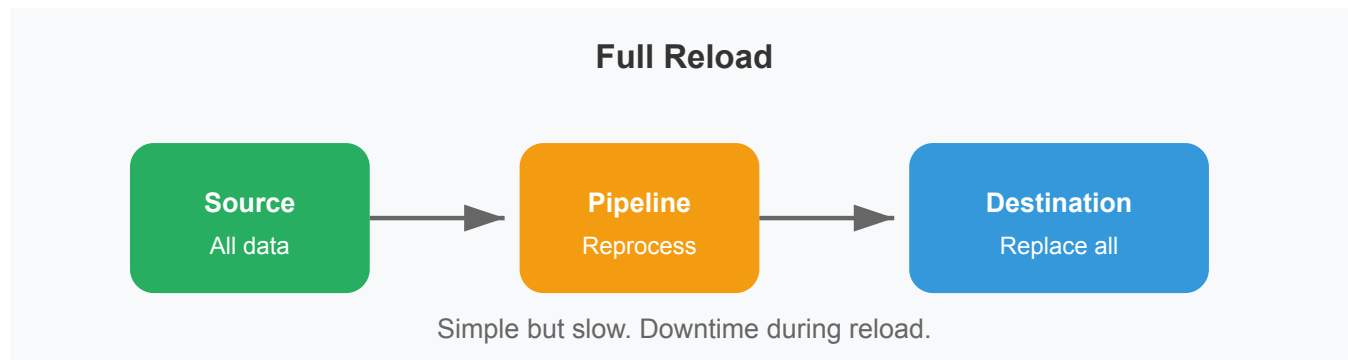


Backfill and Reprocessing Strategies for Data Pipelines

 systemdesignschool.io/fundamentals/backfill-reprocessing



Backfill & Reprocessing

Data pipelines need to rerun. A bug corrupted last month's data. A schema change needs historical records updated. The analytics team wants a new metric computed across all history. In each case, you need to reprocess data that the pipeline already handled.

Reprocessing isn't exceptional—it's expected. Production pipelines regularly need to backfill historical data. The question isn't whether you'll need to do it, but how gracefully your pipeline handles it.

Common Reprocessing Scenarios

Bug fixes. A logic error in your transformation produced incorrect output for the past month. Once you fix the code, you need to reprocess affected data.

Schema evolution. You've added a new field to your data model. Existing records need to be backfilled with values computed from raw data.

New derived data. Analytics wants a metric that didn't exist when the pipeline was built. You need to compute it across historical data, not just new events.

System migration. You're moving to a new storage system or data warehouse. Reprocessing populates the new destination with historical data.

Compliance or audit. A data correction is required for regulatory reasons. You need to update specific historical records.

Strategy 1: Full Reload

The simplest approach: reprocess everything from scratch. Delete the existing output, run the pipeline from the beginning of time, replace all data.

Full Reload



Simple but slow. Downtime during reload.



When it works well:

- Dataset is small enough to reprocess quickly
- Destination can handle complete replacement without breaking downstream consumers
- Simplicity is worth more than speed

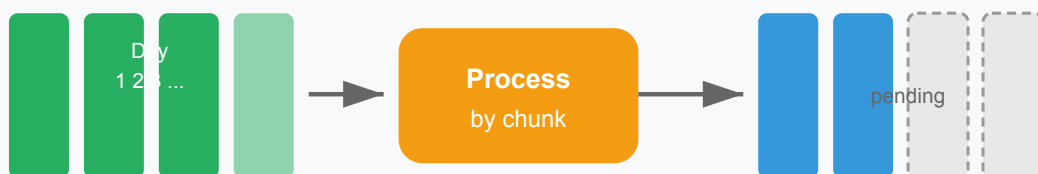
Trade-offs:

- Slow for large datasets—reprocessing years of data takes time
- Destination may be unavailable or inconsistent during the reload
- Expensive in terms of compute resources

Strategy 2: Incremental Backfill

Reprocess in bounded chunks—typically by time partition (day, week, month). Track which chunks have been completed. Resume from failures without starting over.

Incremental Backfill



Process in chunks. Resume from failures. Lower peak load.



Concrete example: Your pipeline partitions output by date. To backfill the last 90 days:

1. List partitions needing reprocessing: 2024-01-01 through 2024-03-31
2. Process each partition independently, writing to its output location
3. Track completed partitions in a checkpoint table
4. If the job fails on day 45, restart from day 45, not day 1

When it works well:

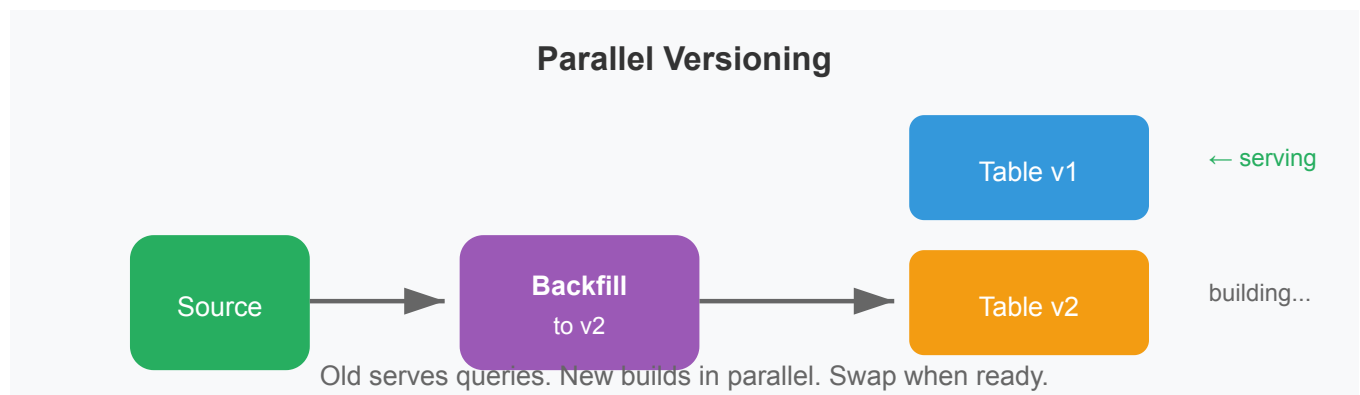
- Large datasets where full reload would take too long
- Need to show progress and provide estimates
- Can't afford extended downtime—data becomes available incrementally

Trade-offs:

- More complex coordination logic
- Must handle partial state—what happens if you crash mid-partition?
- Dependencies between partitions (e.g., slowly changing dimensions) require careful ordering

Strategy 3: Parallel Versioning

Write to a completely new output table while the old table continues serving traffic. Once the new version is complete and validated, atomically swap it in.



Concrete example:

1. Current production reads from `sales_summary_v1`
2. Backfill writes to `sales_summary_v2` (new table, same schema)
3. Run validation queries comparing v1 and v2
4. Once confident, update the view/alias to point to v2
5. Keep v1 around briefly for rollback, then delete

When it works well:

- Zero downtime required
- Need to validate output before exposing it to users
- Rollback capability is important

Trade-offs:

- Double storage during the migration period
- Requires an atomic swap mechanism (view, symlink, table rename)
- Downstream systems must use the indirection layer

The Foundation: Idempotent Writes

Regardless of strategy, the key to safe reprocessing is **idempotent writes**: processing the same input twice produces the same result without creating duplicates or inconsistencies.

Non-idempotent (dangerous):

```
INSERT INTO sales_summary (date, total) VALUES ('2024-01-15', 1500); --  
Reprocessing creates duplicate rows
```

Idempotent (safe):

```
INSERT INTO sales_summary (date, total) VALUES ('2024-01-15', 1500) ON CONFLICT  
(date) DO UPDATE SET total = 1500; -- Reprocessing overwrites, doesn't duplicate
```

For batch pipelines, a common pattern is to partition outputs by time and replace entire partitions atomically:

```
/output/date=2024-01-15/ <- Replace this whole partition  
/output/date=2024-01-16/
```

Each partition is either fully present (old version) or fully present (new version), never a mix. Atomic partition replacement is idempotent—processing twice just overwrites with the same data.

Designing for Reprocessability

Build reprocessing into your pipeline design from the start:

Retain source data. Don't transform away the original. If you only store transformed data, you can't recompute with new logic. The ELT pattern helps here—raw data persists in the warehouse.

Version your transformations. Track which logic version produced each output (a version column, or metadata in the output path). When logic changes, you know which outputs are stale and need reprocessing.

Partition by time. Time-based partitioning makes incremental backfill natural. You can replace one day's data without touching others. It also aligns with typical "reprocess the last N days" workflows.

Test reprocessability. Periodically replay historical data through your pipeline as a test. Catch issues (non-idempotent writes, source data retention problems) before they become urgent production incidents.

What's Next

Reprocessing handles bulk corrections. For individual records that fail during normal pipeline operation, error handling adds **dead-letter queues**, retries, and monitoring strategies for production pipelines.